

Grafici con matplotlib

Dal file di numeri scritto in C al grafico salvato in un file PNG

Shahram Rahatlou

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027

SAPIENZA
UNIVERSITÀ DI ROMA

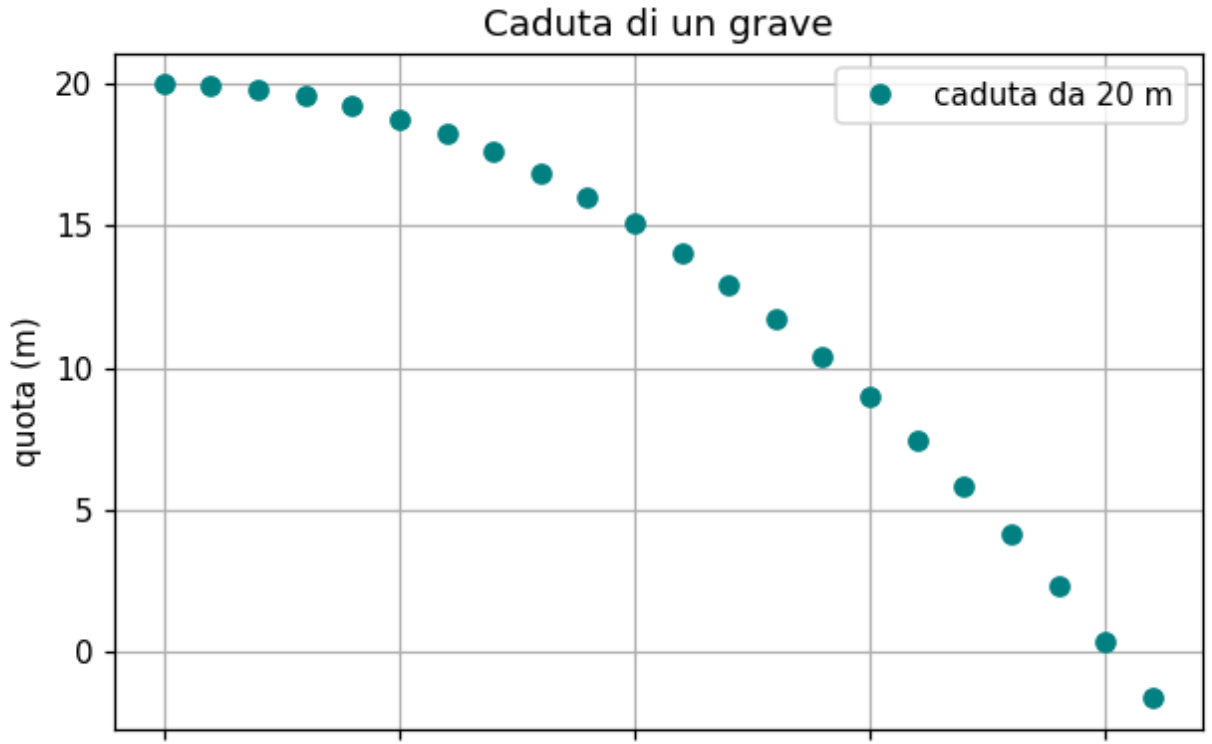
Che aspetto ha la caduta?

Il programma in C ha scritto 22 righe di numeri: un grafico si capisce a colpo d'occhio

grave.txt (colonne t e y)

```
0.00 20.00
0.10 19.95
0.20 19.80
0.30 19.56
0.40 19.22
0.50 18.77
0.60 18.23
0.70 17.60
0.80 16.86
0.90 16.03
   ...
1.90 2.29
2.00 0.38
2.10 -1.63
```

22 righe



La quota scende sempre più in fretta

i punti disegnano una parabola: lo si vede subito, nei numeri no

Dal file a due array

np.loadtxt legge le colonne del file: ogni colonna diventa un array

leggi.py

```
import numpy as np

# prima colonna in t, seconda in y
t, y = np.loadtxt('grave.txt', unpack=True)

print("righe lette: %d" % len(t))
print("t[0] = %.2f  y[-1] = %.2f" % (t[0], y[-1]))
```

Terminale

```
$ python3 leggi.py
righe lette: 22
t[0] = 0.00  y[-1] = -1.63
```

Se nel file c'è testo

```
grave_errato.txt, l'inizio (output del programma con tutto il testo, ./app.exe > grave_errato.txt)
altezza h (m): passo dt (s):    t          y
  0.00   20.00
$ python3 leggi_errore.py
ValueError: could not convert string 'altezza' to float64 at row 0, column 1.
```

Il primo grafico in cinque righe

Leggo il file, disegno la quota contro il tempo, apro la finestra

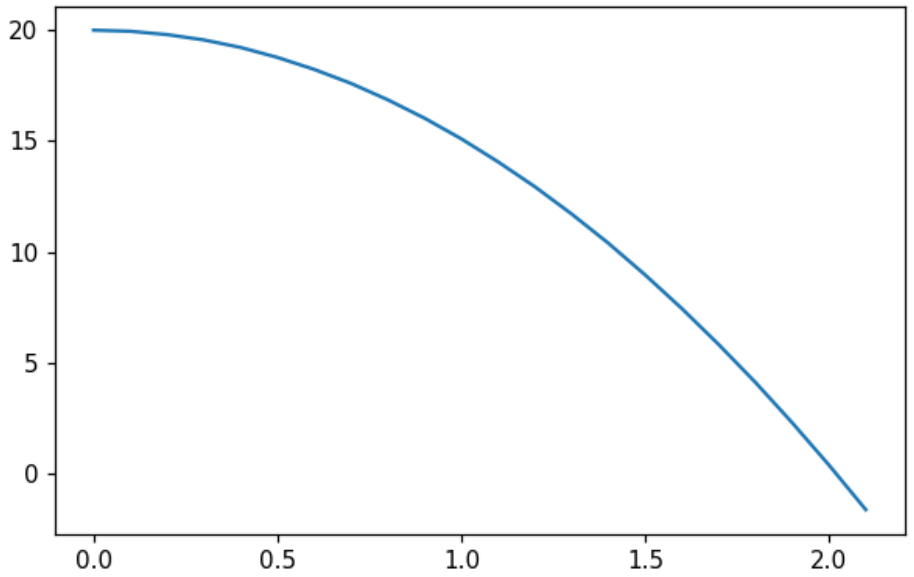
primo.py

```
import matplotlib.pyplot as plt
import numpy as np

t, y = np.loadtxt('grave.txt',
unpack=True)
plt.plot(t, y)
plt.show()
```

Terminale

```
$ python3 primo.py
```



plt.plot(t, y)

disegna y in funzione di t: un punto per riga del file

plt.show()

apre la finestra con il grafico; si chiude con la x

plt, np

i nomi corti dati da import ... as

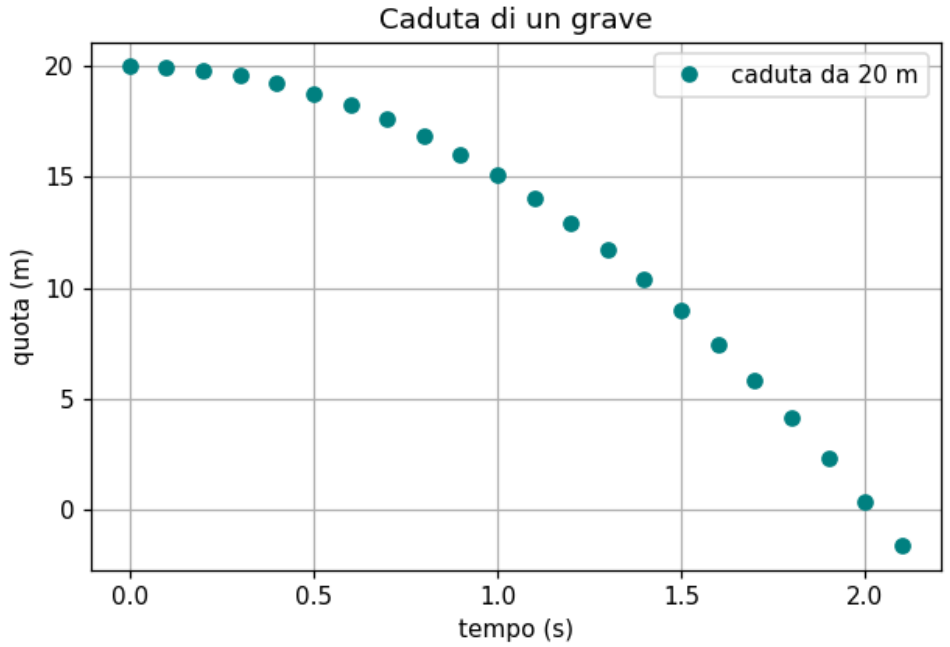
Rendere il grafico leggibile

Che cosa sono gli assi? E i punti? Stile, etichette e legenda

stile.py (in verde le righe nuove)

```
t, y = np.loadtxt('grave.txt', unpack=True)

plt.plot(t, y, 'o', color='teal', label='caduta
da 20 m')
plt.xlabel('tempo (s)')
plt.ylabel('quota (m)')
plt.title('Caduta di un grave')
plt.legend()
plt.grid()
plt.show()
```



'o'

solo punti; 'o-' punti e linea; 'x' croci; '--' tratteggio

color='...'

'red' 'blue' 'green' 'orange' 'black' 'gray' 'teal'

label + legend()

il testo della legenda

xlabel, ylabel

assi con unità; title e grid

Dove cade? Limiti e righe di riferimento

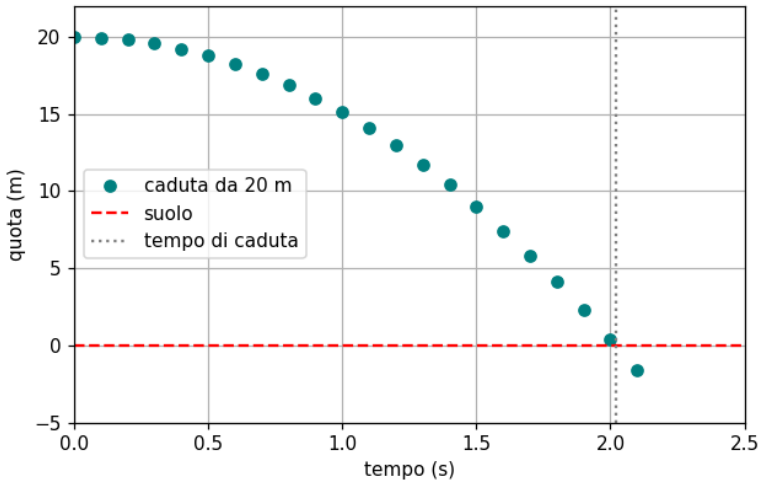
xlim e ylim scelgono la finestra, axhline e axvline segnano un valore, savefig salva il file

limiti.py (in verde le righe nuove)

```
t, y = np.loadtxt('grave.txt', unpack=True)

plt.plot(t, y, 'o', color='teal', label='caduta da 20 m')
plt.xlabel('tempo (s)')
plt.ylabel('quota (m)')
plt.xlim(0., 2.5)
plt.ylim(-5., 22.)
plt.axhline(y=0., color='red', linestyle='--', label='suolo')
plt.axvline(x=np.sqrt(2. * 20. / 9.81), color='gray',
linestyle=':', label='tempo di caduta')
plt.legend()
plt.grid()

plt.savefig('grave.png', dpi=150)    # prima di show!
plt.show()
```



`savefig` prima di `show`: dopo show la figura non esiste più

plt.axhline(y=0.)

riga orizzontale a un valore: il suolo

plt.axvline(x=...)

riga verticale a un valore: il tempo di caduta

plt.xlim, plt.ylim

la finestra, scelta guardando i dati

Dati e formula analitica a confronto

np.linspace crea molti tempi equidistanti; la formula si applica a tutto l'array in un colpo

confronto.py

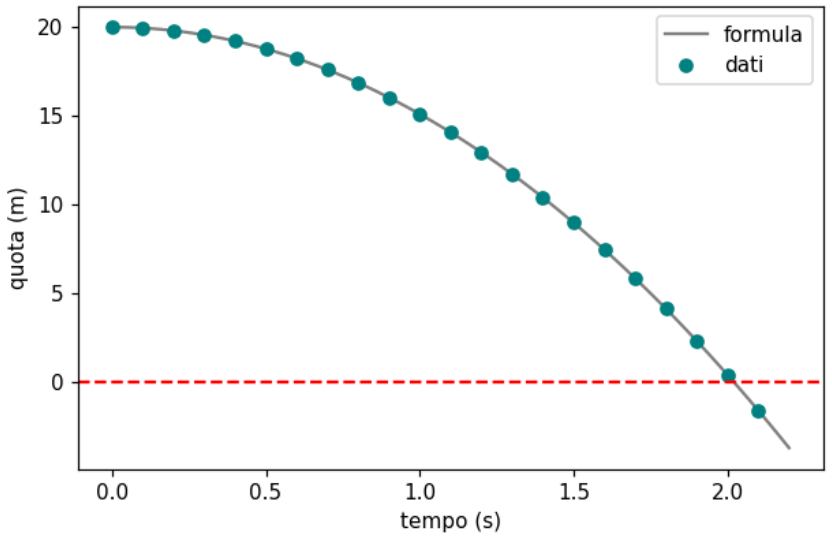
```
import matplotlib.pyplot as plt
import numpy as np

GRAV = 9.81
h = 20.

t, y = np.loadtxt('grave.txt', unpack=True)

# 100 tempi tra 0 e 2.2
tt = np.linspace(0., 2.2, 100)
yy = h - 0.5 * GRAV * tt**2     # formula su tutto l'array

plt.plot(tt, yy, '-', color='gray', label='formula')
plt.plot(t, y, 'o', color='teal', label='dati')
plt.axhline(y=0., color='red', linestyle='--')
plt.xlabel('tempo (s)')
plt.ylabel('quota (m)')
plt.legend()
plt.show()
```



np.linspace(0., 2.2, 100)

100 valori equidistanti tra 0 e 2.2: la curva liscia, anche tra un punto e l'altro e sotto lo zero

Scala logaritmica: quando i numeri crescono molto

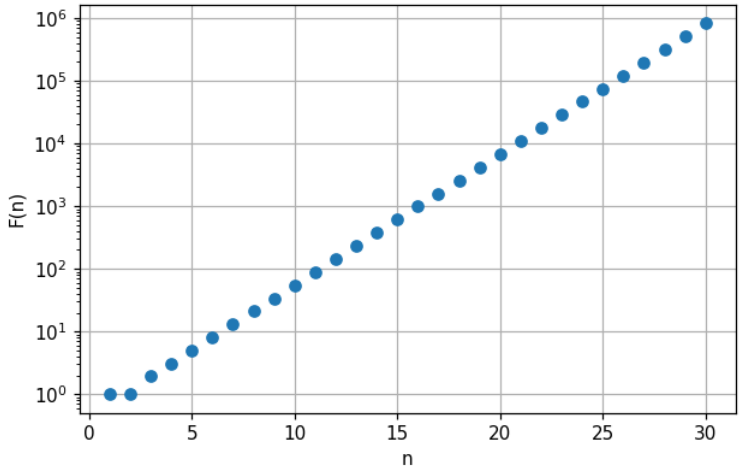
fibonacci.txt, scritto dal programma in C: n, F(n) e il rapporto F(n+1)/F(n)

fibo_log.py

```
import matplotlib.pyplot as plt
import numpy as np

n, fib, rapporto = np.loadtxt('fibonacci.txt', unpack=True)

plt.plot(n, fib, 'o')
plt.yscale('log')
plt.xlabel('n')
plt.ylabel('F(n)')
plt.grid()
plt.show()
```



plt.yscale('log')

asse y logaritmico: F(n) diventa una retta, segno di una crescita esponenziale

fibonacci.txt (n, F(n), rapporto)

```
1 1 1.000000
2 1 2.000000
3 2 1.500000
...
30 832040 1.618034
```

Il rapporto e la sezione aurea

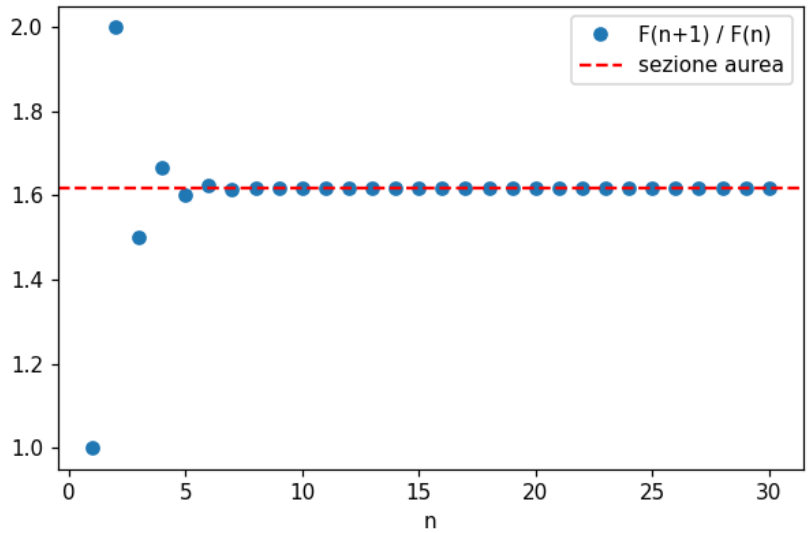
F(n+1)/F(n) si avvicina a (1 + √5) / 2 = 1.618034

fibo_rapporto.py

```
import matplotlib.pyplot as plt
import numpy as np

n, fib, rapporto = np.loadtxt('fibonacci.txt', unpack=True)

aurea = (1. + np.sqrt(5.)) / 2.
plt.plot(n, rapporto, 'o', label='F(n+1) / F(n)')
plt.axhline(y=aurea, color='red', linestyle='--',
label='sezione aurea')
plt.xlabel('n')
plt.legend()
plt.show()
```



aurea = (1. + np.sqrt(5.)) / 2.

np.sqrt è la radice quadrata di numpy (M08); il valore è 1.618034

plt.axhline(y=aurea, ...)

una riga orizzontale a un valore: dopo pochi passi i punti ci stanno sopra

Un grafico a barre

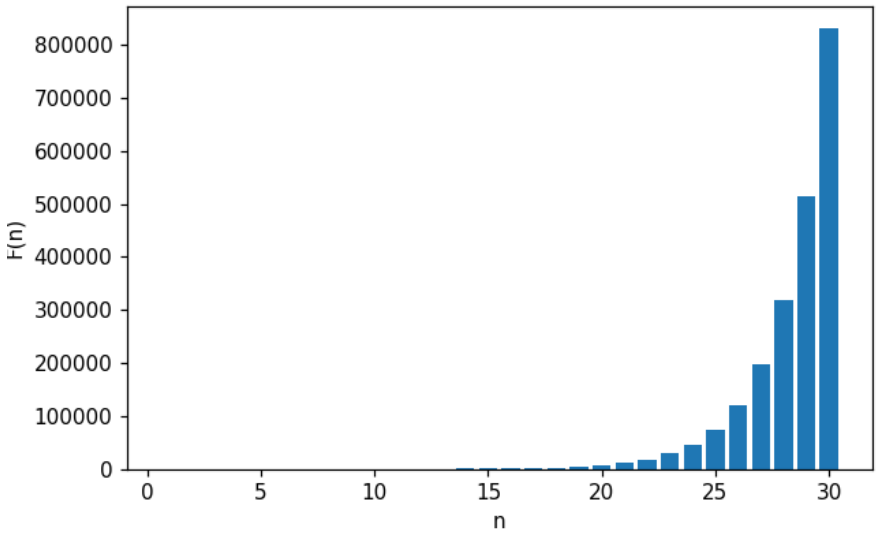
plt.bar disegna una barra per ogni valore: qui i 30 numeri di Fibonacci

fibo_barre.py

```
import matplotlib.pyplot as plt
import numpy as np

n, fib, rapporto = np.loadtxt('fibonacci.txt', unpack=True)

plt.bar(n, fib)
plt.xlabel('n')
plt.ylabel('F(n)')
plt.show()
```



`plt.figure()`

apre una figura nuova: serve un grafico per figura se un programma ne disegna più di uno

plt.bar(n, fib)

una barra per ogni coppia (n, F(n)): come plt.plot, ma con le barre

Barre in scala logaritmica

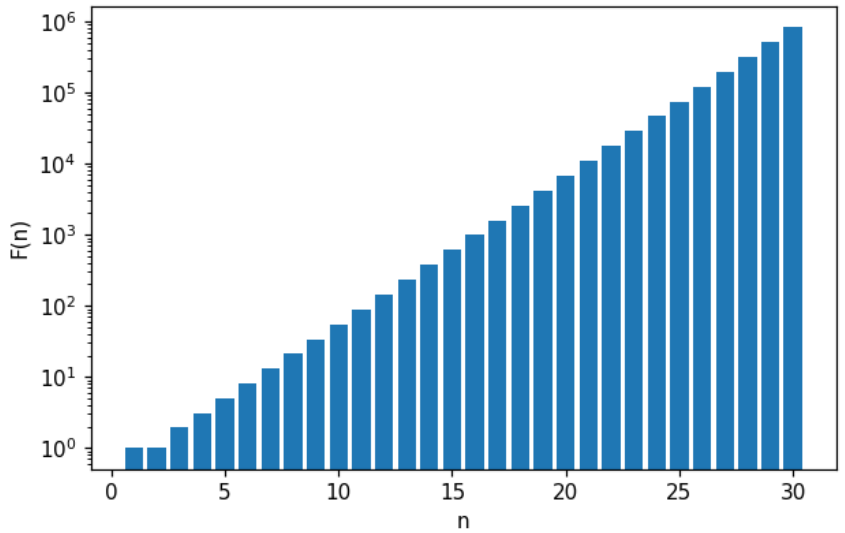
Lo stesso grafico a barre, con l'asse y logaritmico

fibo_barre_log.py (in verde la riga nuova)

```
import matplotlib.pyplot as plt
import numpy as np

n, fib, rapporto = np.loadtxt('fibonacci.txt', unpack=True)

plt.bar(n, fib)
plt.yscale('log')
plt.xlabel('n')
plt.ylabel('F(n)')
plt.show()
```



`plt.yscale('log')`

anche con le barre: ora si vedono tutte, dalla prima all'ultima

Confronta con la slide 10

le barre ora crescono a gradini quasi uguali: è la retta della scala logaritmica

Un modello da copiare nelle lab

grafico.py: leggi, disegna, etichetta, aggiungi un riferimento, salva, mostra

grafico.py

```
import matplotlib.pyplot as plt
import numpy as np

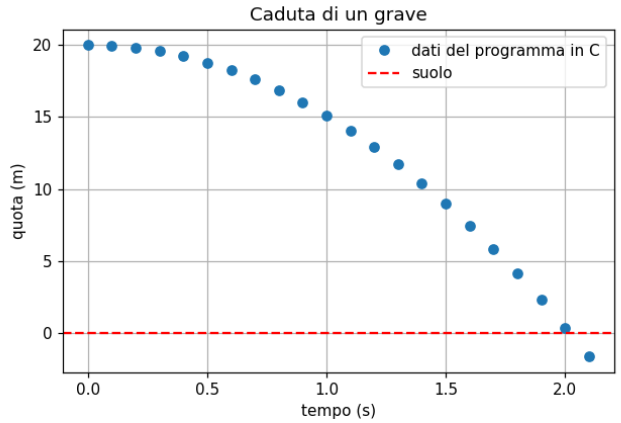
# dati: una colonna del file = un array
x, y = np.loadtxt('grave.txt', unpack=True)

# il grafico: i dati sono solo punti
plt.plot(x, y, 'o', label='dati del programma in C')

# etichette, titolo, griglia (senza accenti)
plt.xlabel('tempo (s)')
plt.ylabel('quota (m)')
plt.title('Caduta di un grave')
plt.grid()

# un riferimento: linea orizzontale a y = 0, con il suo nome per la legenda
plt.axhline(y=0., color='red', linestyle='--', label='suolo')
plt.legend()   # dopo tutte le curve e le righe, altrimenti manca 'suolo'

# salvare PRIMA di mostrare
plt.savefig('grafico.png')
plt.show()
```



Cambia solo

nome del file, colonne, etichette

plt.savefig(...)

salva il grafico nel file grafico.png, che trovi nella cartella dopo python3 grafico.py. Va scritta PRIMA di plt.show()

Esercizi

Da fare a casa, poi si controlla con lo script della soluzione

1 Il pendolo

Disegna l'angolo in funzione del tempo: solo punti, titolo, assi con unità, griglia e legenda. Salva in pendolo.png.

dati:
`pendolo.txt (t, theta)`

2 Interessi

Capitale in funzione degli anni, con asse y logaritmico, la curva della formula 1000 · 1.035^anno e una linea rossa a 2000 euro.

dati:
`capitale.txt (anno, euro)`

3 Seno e coseno

Disegna sin(x) e cos(x) per x da 0 a 2π con 200 punti (np.linspace, np.pi), una riga orizzontale a y = 0 e una verticale a x = π. Titolo, assi, legenda.

dati:
`nessun file, solo linspace`

Esegui con `python3 nome.py` (i file .txt sono quelli scritti dai programmi in C di M07)

Riepilogo

1	**Leggere**	np.loadtxt('file.txt', unpack=True): una colonna del file, un array; solo numeri nel file
2	**Disegnare**	plt.plot con formato, label, legend; xlabel, ylabel, title, grid; xlim, ylim, axhline, axvline; yscale('log')
3	**Salvare e mostrare**	plt.savefig('nome.png') PRIMA di plt.show(); plt.bar per le barre. Istogrammi (hist) e scatter dopo i numeri casuali